

Turning IDN edge cases into typosquats

Turning IDN edge cases into typosquats - Ian Muscat, Leanne Briffa, Have I Been Squatted.

- Published: 2026-10-03
- Original: <<https://haveibeensquatted.com/blog/turning-idn-edge-cases-into-typosquats>>
- Preserved from: <https://haveibeensquatted.com/blog/turning-idn-edge-cases-into-typosquats> (live) on 2026-10-03
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so it remains readable if the page goes offline. To read the original, follow the link above.

Content

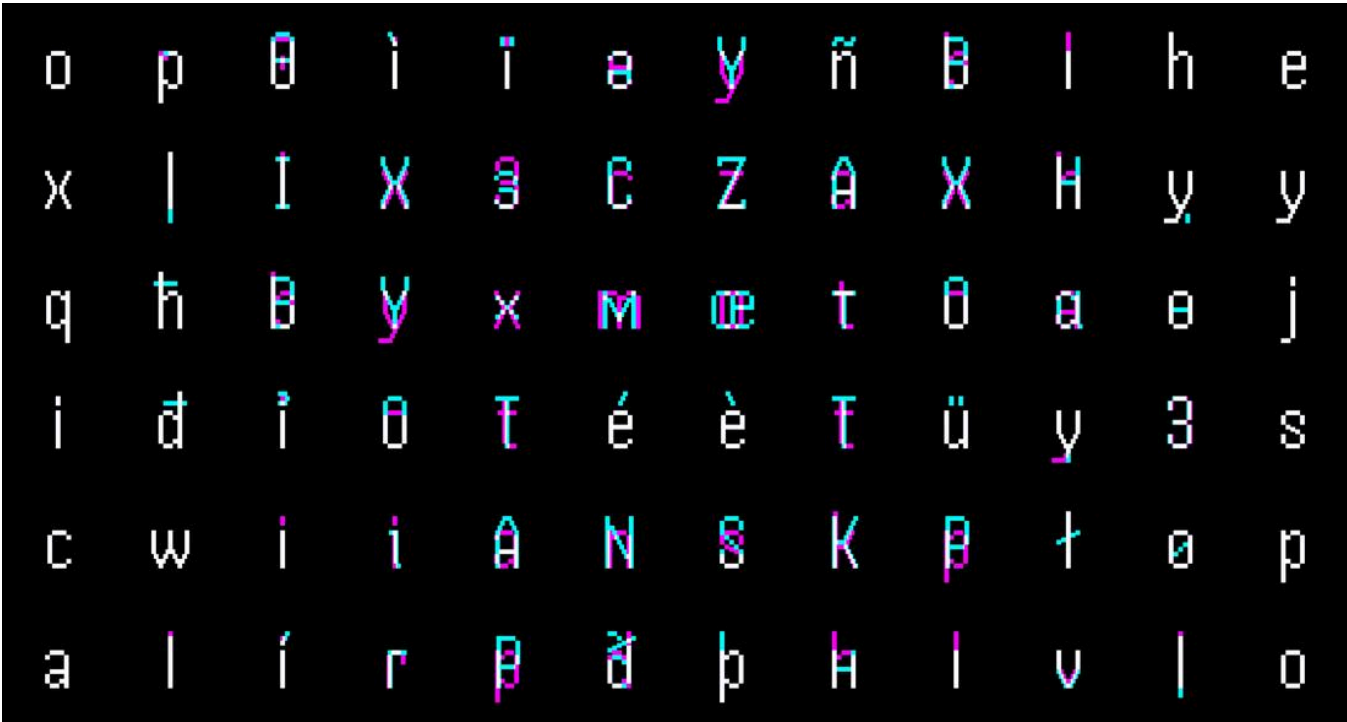
UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

Turning IDN edge cases into typosquats

October 3, 2026

ResearchHomograph attacksEmail security

by Ian Muscat and Leanne Briffa · 40 min read



Overview

"Check the address bar" is the one piece of security advice everybody has heard. It assumes that the name shown there is the name of the site. For most of the internet's history that was safe, because domain names could only contain the 26 letters of the English alphabet, ten digits, and a hyphen. Since 2003 they can contain letters from almost any writing system, so a bookshop in Berlin can be bücher.de and a newspaper in Athens can spell its name in Greek. Many of those letters, however, look identical to English ones. Cyrillic has its own a, e, o, p, and c, drawn the same way in most fonts as Latin a, e, o, p, and c, but different characters to a computer. A name can therefore read as apple.com and not be it.

Browsers account for this. Chrome, which this post examines most closely, applies a display check. If a name looks as though it was built to deceive, the address bar shows its encoded form, such as xn--80ak6aa92e.com, instead of the lookalike. Mail clients make their own choice about how to draw a sender's address, and the registries that sell domain names decide separately which letters a name may contain. Three systems make three separate decisions, and none consults the others.

The question this post asks is whether someone can still register a domain that looks like a well-known one and have Chrome draw it as the real thing. The answer is yes, and the clearest example is one Chrome already fixed once.

In April 2017, Xudong Zheng registered apple.com. Every character before .com is Cyrillic, and Chrome, Firefox, and Opera all drew it as something almost indistinguishable from apple.com. Browsers at the time hid the Unicode form of a name only when its characters came from more than one writing system, and Zheng showed that a name built entirely from one foreign script was not caught. Chromium responded with a rule that has held since: a label written entirely in Cyrillic letters that look like Latin ones is shown as its raw xn-- encoding instead.

Illustration

Compare Latin and Cyrillic domain names

apple.com

apple.com

Cyrillic name as an A-label

xn--80ak6aa92e.com

Top row: Latin apple.com. Bottom row: every letter before .com replaced with a Cyrillic lookalike. Latin a is U+0061; Cyrillic a is U+0430. Select a character to see its name and compare fonts.

Nine years later, that name shows as Punycode. A name one character away does not. Replace the final e with ø, a Cyrillic letter used in Kazakh, Mongolian, and Tatar, and the result is appleø.com. Have I Been Squatted registered it. Chrome shows it in Unicode. A visitor who signs in to apple.com every day and lands on it sees no warning of any kind.

The figure below draws it, and the other names registered for this research, at the size a browser draws its address bar.

Illustration

At address-bar size

Lookalike

Original

apple.com

apple.com

openai.com

openai.com

skype.com

skype.com

sharepoint.com

sharepoint.com

pinterest.com

pinterest.com

Show 15 more

Each row pairs a registered lookalike with the name it imitates, drawn at 14 pixels in the system interface font, close to how a browser draws its address bar. The glyphs depend on the fonts installed on the viewing device.

The rest of this post follows that one name through every layer that could have stopped it: Chromium's display check, the skeleton comparison behind it, the warnings Chrome shows when a page loads, Verisign's registry tables, the .com zone, and finally Gmail and Outlook, where the same domain arrives as a sender address. None of the layers is broken. Each makes a defensible decision from the information it has, and that independence is the finding. The Chromium team does not treat any of this as a vulnerability, and this post agrees. A check that hid every lookalike would also hide the ordinary names that readers in Kazakh, Vietnamese, or German need. The scope is deliberately narrow. Chromium is the engine behind Chrome, Edge, and other Chromium-based browsers; Firefox and Safari apply their own policies and were not tested, and neither were Apple Mail or iOS Mail. Homograph attacks of this kind were first described by Gabrilovich and Gontmakher in "[The Homograph Attack](#)" in 2002.

Background: Unicode, Punycode, and IDNA

How Chromium decides what to draw

When Chromium is about to show xn--80a6aa68c8d.com in the address bar, it has to choose between that A-label and the U-label apple.com. The policy is built on [Unicode Technical Standard #39 \(UTS #39\)](#), which specifies which scripts may be mixed, which characters count as confusable, and how to reduce a string to a comparison "skeleton". Chromium runs the first part of the check through International Components for Unicode (ICU), configured with a narrowed set of allowed characters, and then applies rules of its own.

The decision has two stages, and both must pass. The first runs per label: each xn-- label goes through `SafeToDisplayAsUnicode()` with the top-level domain (TLD) as context, and plain ASCII labels skip it. The second looks at the whole hostname: `GetSimilarTopDomain()` reduces it to a skeleton and compares that against a bundled list of 8,462 popular domains. A match anywhere sends the entire hostname back to Punycode, even if every label passed the first stage.

Counted the way the source counts them, that is seven per-label checks and one whole-hostname check. Three of the eight do nearly all the work: the ICU check that enforces the allowed-character set and forbids mixing scripts, the whole-script confusable rule that stopped apple.com, and the skeleton comparison against the bundled list. Three more handle single characters that are safe only under one TLD, and one is a guard in the code that real input never triggers. The figure below runs every spelling of apple that the research's substitution map can build, with ø added for e, through all eight in the order Chromium runs them, and follows apple.com. Select a check to see what it removes, a name it blocks, and why apple.com gets through.

Model result

Every spelling of apple, through eight checks

1 . ICU spoof

18,719 in · 443 left

Following · passes

apple.com

All five letters are Cyrillic, and each is in Chromium's allowed set.

Blocked here · Punycode

xn--pple-43d.com

From apple.com

One Cyrillic a in front of Latin pple mixes scripts.

This check does two things simultaneously.

Script mixing detection. Every Unicode character belongs to a "script" (Latin, Cyrillic, Greek, and so on). Chromium configures ICU's spoof checker with `USP00F_HIGHLY_RESTRICTIVE`, ICU's name for the [Highly Restrictive](#) level from UTS #39, via [u](#)

`spoof_setRestrictionLevel()` in `idn_spoof_checker.cc`. That setting rejects labels that mix scripts in unsanctioned ways.

For instance, one Cyrillic e inside an otherwise Latin word such as email will cause Chromium to show Punycode, whether or not the characters look alike. The only mixes permitted pair Latin with a Chinese, Japanese, or Korean (CJK) writing system, because those languages routinely write Latin words alongside their own scripts. This is why `apple.com` has to be Cyrillic all the way through, since a single Latin letter in it would fail here.

Character allowlist. Not every Unicode character is even eligible to appear. Chromium maintains a curated set of "allowed" characters starting from ICU's recommended and inclusion sets, then removing entire blocks and individual code points known to be dangerous, such as Pinyin `ǐ`, dotted `đ` from the start of Latin Extended Additional, and Greek `ϖ`. If a label contains any character outside this set, even one that is valid Unicode and used in some language, Chromium rejects the whole label without checking anything else. `ø` is in the set.

Inspect the allowed character set

2 . Icelandic

443 in · 443 left

Following · passes

`apple.com`

The label has no `þ` or `ð`.

Blocked here · not an apple spelling

`xn--orn-ooa.com`

From `þorn.com`

`þ` is allowed only under `.is` and `.fo`.

Not every character is safe under every TLD. Chromium allows Icelandic thorn `þ` and eth `ð` only on `.is` and `.fo`. On any other TLD, a label that contains either character is shown as Punycode.

3 . Latin schwa

443 in · 443 left

Following · passes

`apple.com`

The label has no `ə`.

Blocked here · not an apple spelling

`xn--kbab-v6b.com`

From `kəbab.com`

ə is allowed only under .az.

Chromium allows Azerbaijani schwa ə only on .az . On any other TLD, a label that contains it is shown as Punycode.

4 . Middle dot

443 in · 443 left

Following · passes

appleø.com

The label has no middle dot.

Blocked here · not an apple spelling

xn--collegi-xma.com

From col·legi.com

The middle dot is allowed only as l·l under .cat.

Chromium allows the middle dot · only as the Catalan ela geminada l·l on .cat . On any other TLD, or in any other pattern, the label is shown as Punycode.

5 . ASCII

443 in · 443 left

Following · continues

appleø.com

The label is not ASCII, so the remaining checks still run. A pure-ASCII label is safe and returns early, so nothing is blocked here.

No label is blocked at this check.

If ICU reports that the label contains only ASCII characters, Chromium treats it as trivially safe and skips the remaining per-label checks. A valid xn-- label always decodes to at least one non-ASCII character, and a plain label such as example never enters

SafeToDisplayAsUnicode() at all, so in practice this is a guard in the code rather than a filter that decides real names.

6 . Whole-script

443 in · 441 left

Following · passes

appleø.com

ø is not on the Cyrillic lookalike list, so the whole-script rule stands down. The label is single-script, so the function returns safe here.

Blocked here · Punycode

xn--80ak6aa92e.com

From apple.com

Every letter is on the lookalike list, and .com is not exempt.

The whole-script confusable (WSC) check is the rule that caught apple.com. It targets a label written in a single script, such as Cyrillic, in which every character is a Latin confusable.

Chromium maintains a list of 29 Cyrillic characters that resemble Latin letters (a for а, e for е, o for о, p for р, c for с, and so on). If the label is single-script Cyrillic and every character is on this list, Chromium concludes that this is Latin impersonation and shows Punycode. Some legitimate Cyrillic words happen to be composed entirely of such characters, so Chromium also keeps a hardcoded allowlist of 12 words, including парк (park), театр (theater), and курс (course).

Inspect the Cyrillic lookalike list

The same logic applies to 16 other scripts, such as Armenian, Greek, and Georgian, each with its own list. A script's check also stands down under TLDs that legitimately use that script. Cyrillic gets a pass on .ru, .ua, and .bg, among others, Greek on .gr, and Armenian on .am. The Cyrillic list is the largest and the most relevant, because it covers enough Latin letters to spell whole English words. Chromium added м to it on 11 September 2026, taking it from 28 to 29 characters. That landed after Chrome 155 branched, so Chrome 155 still ships the 28-character list and a later release carries the 29th.

A digit lookalike check runs at the same point. If a label contains only digits and characters that look like digits (0 for 0, 3 for 3), Chromium shows Punycode for names such as 1234.com.

The rule is all-or-nothing. If even one Cyrillic character in the label is not on the list, this check does not reject the label. 0 is not on the list.

7 . Patterns

441 in · 392 left

Following · not run

apple.com

A single-script label without combining marks has already returned at check 6.

Blocked here · Punycode

xn--appe-df5f.com

From app[]e.com

Latin with Han passes check 1, but [] is a CJK lookalike next to Latin letters.

This check handles the edge cases that checks 1 and 6 do not cover. It sees labels that mix scripts in a permitted way, such as Latin with Japanese, and single-script labels that contain combining marks. Chromium looks for patterns such as the following.

- **Non-ASCII Latin mixed with non-Latin.** A label with accented Latin characters *and* characters from a script outside the Latin-Greek-Cyrillic family is considered suspicious.
- **Combining mark tricks.** A combining dot above ◌ placed after i, j, or l can fake other characters, like turning l into something that looks like i.
- **Katakana slash lookalikes.** Katakana ゴ, ノ, ヲ, and ヱ resemble / and could fake path separators.
- **CJK lookalikes at a boundary.** CJK characters that resemble Latin letters or punctuation, such as ー (a hyphen) and 〇 (an o), placed next to non-CJK characters.

Most labels never reach this check, because the single-script path in check 6 already decides them. `apple` is single-script with no combining marks, so it returns safe before this point.

8 . Top domains

392 in · 99 left

Following · passes

`apple.com`

The skeleton, `apple`, matches nothing in the bundled table. `apple.com` is in it, but its skeleton is `apple`.

Blocked here · Punycode

`xn--appl-epa.com`

From `applé.com`

The accent disappears, and the skeleton matches `apple.com`.

Stage 2 builds the skeletons for the whole hostname in four steps.

- **Strip diacritics.** The label is decomposed into base letters and combining marks ([Unicode Normalization Form D](#)), the marks are removed, and the result is recomposed. `sérvice` becomes `service`, and `bücher` becomes `bucher`.
- **Generate variants for ambiguous characters.** `l` could be an `i` or an `l`, so Chromium generates a variant for each reading.
- **Map confusables to ASCII.** Chromium's own additions to the Unicode list convert lookalikes such as `q` to `p`, `h` to `h`, and `k` to `h`.
- **ICU skeleton computation.** A final pass applies the UTS #39 confusable mappings to each variant and produces the comparison strings.

Each resulting skeleton is looked up against Chromium's [bundled domain list](#) of 8,462 names, 836 of them in a higher-priority top bucket. At build time, that list is turned into a [skeleton table](#), which is what the runtime check searches. If a generated skeleton matches an entry for a different site, Punycode is displayed for the entire hostname. `apple.com` is in the top bucket. The skeleton of `apple.com` is not `apple.com`, for a reason the next two sections explain.

25,919 spellings of apple built from the research's substitution map, with ø added for e. 7,200 fail IDNA encoding before Chromium sees them. Of the 99 that display as Unicode, 98 swap l for ĩ or j, whose skeleton reads appie, and one is applø.com.

Chromium source files for the eight checks

Chromium's [IDN policy](#) states the trade-off directly.

“ We want to prevent confusion, while ensuring that users across languages have a great experience in Chrome. Displaying either punycode or a visible security warning on too wide of a set of URLs would hurt web usability for people around the world. ”

— Chromium IDN policy

Names like applø.com exist inside that trade-off. To measure how much room it leaves, Have I Been Squatted built a Python replica of the display check using PyICU, which makes the same ICU calls Chromium makes. It runs ICU 78, the release line Chrome has shipped since version 148, and matches a 33-case subset of the test vectors in Chromium's `idn_spoof_checker_unittest.cc`. A candidate in the rest of this post is a spelling of a target name with one or more letters replaced from a substitution map of Latin and Cyrillic lookalikes; the search keeps candidates that encode under IDNA and pass the full display pipeline, and stops after 500 accepted per name.

The breaker: one character off the list

The whole-script rule is all-or-nothing, so one Cyrillic character that is *not* on the lookalike list keeps the whole label out of it. This post calls that character a breaker. A useful one has to satisfy three conditions at once.

- It sits in Chromium's allowed Unicode set, so it clears check 1.
- It is missing from the Cyrillic lookalike list, so check 6 does not fire.
- It still looks enough like a Latin letter that a reader can mistake it for the letter it replaces.

Under ICU 78 and the 29-character list, 64 lowercase Cyrillic characters clear the first two conditions. The third is a visual question, not a Unicode one, and this research draws on Paul Wood's [confusable-vision](#) project, whose [RaySpace method](#) compares the vector outlines of two glyphs across system fonts ([release 2026.09.25](#)). Of the 64, only γ clears RaySpace's suggested threshold. The table shows five characters that resemble a Latin letter. The first three are among the 64; the last two were allowed before Chrome 148.

Character	Code point	Resembles	Chrome 148 and later	γ	U+04AF	y	Allowed									
ø	U+04E9	o, e	Allowed	ï	U+0457	i	Allowed	p̣	U+048F	p	Rejected	x̣	U+04FF	x	Rejected	

[Unicode 17.0](#) moved p̣ and x̣ from `Recommended` to `Uncommon_Use`, and ICU's recommended set follows that table. Chrome 148, released in May 2026, moved to ICU 78.2, which carries Unicode 17, so both characters now fail check 1. Chrome 147 and earlier accept all five.

This is the mechanism behind the name in the introduction. Every character in apple.com is on the lookalike list, so the whole-script rule rejects it. Swap the final e for ø, which is not on the list, and the replica returns the opposite verdict.

```
from idn_checker import get_checker

get_checker().safe_to_display_as_unicode("apple", "com", "com")
# IDNSpoofCheckerResult.kWholeScriptConfusable -> Chromium shows punycode

get_checker().safe_to_display_as_unicode("applø", "com", "com")
# IDNSpoofCheckerResult.kSafe -> Chromium shows Unicode
```

Have I Been Squatted registered applø.com along with 14 other Cyrillic names that use ø for e, among them openai.com for openai.com, skypeø.com for skype.com, and shell.com for shell.com; the address-bar figure in the introduction draws them. Each hosts the Have I Been Squatted landing page, and Chrome shows each in Unicode.

The automated search found none of them. Its substitution map, built from RaySpace pairs, links ø only to o, so for apple.com it returns 22 Latin candidates and no Cyrillic ones. The ø-for-e pairing was chosen by eye, and it is the one that works against top-bucket names. The difference between measured glyph similarity and what a reader accepts recurs throughout this post.

The limits of that claim are these. ø keeps the round body and crossbar of e but closes the opening, and RaySpace scores the pair as alike in Arial only. No reader study was run. Whether people miss the closed bowl of ø, one missing dot, or two strokes at address-bar size is a claim this post makes from the glyphs in the figure above, not from measurement.

Two more conditions decide whether a breaker is useful. The whole label has to be written in Cyrillic, and its skeleton must not match a name in the bundled list. The first rules out many words. d, f, g, m, u, v, and z have no Cyrillic substitute in Chromium's allowed set, and Chrome 148 also removed the substitutes for q and w. Every letter in apple has a Cyrillic form, which is why applø.com is possible at all. A Cyrillic spelling of invoice.com would have to keep Latin v, so it mixes scripts and fails check 1 before any breaker matters. Greek has the same whole-script rule with far less room: only γ and χ are both off its list and in Verisign's .com Greek table, so this post does not treat Greek as a practical route.

The second condition holds for most names, because the bundled list covers only popular domains. apple.com and openai.com are both in it, so their lookalikes need a breaker that leaves a trace in the skeleton. That is where ø and γ differ. The figure below puts these conditions together. Pick a word and swap a character for its breaker to see which check decides the outcome.

Model result

One character off the list

apple .com openai .com happy .com invoice .com

-

aaU+ 0430 on the list

- ppU+ 0440 on the list swap
- ppU+ 0440 on the list swap

-

IIU+ 04CF on the list

- eeU+ 0435 on the list swap

Check 6 · whole-script confusable → address bar shows Punycode

xn--80ak6aa92e.com

Every character is on the lookalike list.

Select a letter with more than one form to swap it. The replica's verdicts match Chrome 148 and later; in Chrome 147 and earlier, p also worked as a breaker. apple.com and openai.com are both in the bundled table's top bucket.

What survives a skeleton

Whether Chromium catches a lookalike at stage 2, and what the navigation throttle does with it afterward, both depend on what happens to the substitute character when the skeleton is built. Some substitutions vanish. Others leave a trace. This mechanism separates apple.com from the accented Latin lookalikes that Chromium catches, and it decides most of the outcomes in this post.

Accents are the simple case. The ó in ókta.com is an o with an acute accent, and the accent is removed when Chromium strips diacritics. What remains is okta.com, an exact match for the real name. An accented spelling of a bundled-list name, such as googlé.com, is therefore shown as Punycode.

A hook or a stroke on a letter behaves differently. k̃ is stored as one character rather than a letter plus a separate mark, so diacritic stripping has nothing to remove. The confusable mapping that runs afterward rewrites it as k followed by a combining mark', and because stripping has already happened, the mark stays in the skeleton. ok̃ta.com ends up one character away from the real name instead of matching it. Cyrillic ɵ works the same way, becoming o plus a combining bar-. In appleɵ.com it stands in for e, so the skeleton, appleɵ, differs from apple in two places, the o and the bar. That is why the name escapes the bundled list even though apple.com is in the top bucket. copy.com, by contrast, reduces to exactly copy, because y maps to a plain y; it passes display only because copy.com is not on the list.

Model result

The skeleton drops an accent and keeps a hook.

Acute accentHook on the letterBarred Cyrillic letter

Original letter

ó

After diacritic removal

o

In the skeleton

o

The first step separates the accent in ó from the base letter and removes it. The later steps keep o.

Original domain

ókta.com

Its skeleton

okta.com

Target's skeleton

okta.com

Exact match. The accent leaves no difference.

Select a character to inspect its identity.

Recorded skeletons from the September 2026 model. These strings are used for comparison; DNS still uses the original domain. A dotted circle makes a combining mark visible when shown on its own.

The same mechanism is what makes Latin lookalikes work. Chromium's whole-script check covers 17 scripts, and Latin is not among them. That is a design decision, not an oversight. Latin diacritics are how German, French, Spanish, Portuguese, Polish, Turkish, and Vietnamese are written, and a whole-script rule for Latin would flag bücher.de and café.com alongside the lookalikes. So an accented Latin label passes script mixing (single script), has no whole-script rule to meet, and never reaches the dangerous-pattern check (no combining marks). The skeleton comparison is the only barrier, and it protects only the 8,462 names in its list.

That makes Latin the wider surface by volume. A Cyrillic lookalike needs a substitute for every letter in the label; a Latin lookalike needs one. For copy.com, three Cyrillic candidates pass against 50 Latin ones; for invoice.com, whose v has no Cyrillic form, 329 Latin candidates pass and no Cyrillic ones do. The research's map has a Latin substitute for 19 of the 26 letters, none for b, f, m, p, q, v, and x, although Chromium's allowed set does include hooked letters such as ô, f̂, and v̂ that the map leaves out. Many of the 68 Latin substitutions carry an accent a reader can see. The ones that matter are the ones that differ by a small mark, such as ı for i, which encodes invoice.com as xn--nvoice-o9a.com and displays as written because invoice.com is not on the list.

okta.com

okta.com

The Latin gap and the Cyrillic breaker are therefore the same finding from two sides. What Chromium catches is an exact skeleton match against a short list. What gets through is anything whose skeleton differs from the target, whether because the target is not on the list or because the substitute leaves a mark behind. The number of marks left behind decides what happens next.

Unicode display is not the last check. When a page loads, a separate navigation throttle, `LookalikeUrlNavigationThrottle`, can block the navigation or warn about it even when the address bar shows Unicode. It can show an interstitial, a full-page warning that asks whether the visitor meant the real site, or a "Safety Tip" bubble next to the address bar that does not block the page.

The throttle builds the same skeletons as the display check and compares them against two lists.

- **Sites the visitor uses regularly.** Chromium scores the frequency of visits to each domain, and any domain visited at medium frequency or above counts. An exact skeleton match against one of these sites shows the interstitial.
- **The bundled list of 8,462 domains.** An exact match against one of the 836 top-bucket domains shows an interstitial; against any other domain on the list, a Safety Tip.

When no skeleton matches exactly, Chromium tries two looser comparisons: a skeleton one edit away from a known domain (one character added, removed, or replaced), and a skeleton in which two neighboring characters have swapped places. It runs these only when *both* names have at least five characters before the TLD, counted in ASCII form. The lookalike always clears that bar, because its A-label starts with `xn--`. The target does not always. Against an engaged site, either kind of near match shows a Safety Tip; against the top bucket, only a swap does, and a one-edit match is recorded in metrics but not shown.

The table summarizes what the model predicts for the names in this post, and the figure below runs the comparisons in order for each of them. `proceed` means neither warning is expected.

	Lookalike	Skeleton against target	Visitor who uses the real site	First visit		
googl�.com	Exact match, top bucket	Interstitial	Interstitial			�kta.com
	Exact match	Interstitial	Proceed			okta.com
	One edit, target four chars	Proceed	Proceed			apple.com
	Two edits	Proceed	Proceed			

Model result

The first matching check decides the warning

Profile

No engagement with the real siteUses the real site

Candidate

apple.comokta.com kta.comgoogle.com

Skeleton of the candidate and of apple.com

Candidateapple.comTargetapple.com

2 edits

The barred letter stands in for e, so the skeleton ends in o plus a bar, two edits from apple. The near-match comparisons allow one.

- 1

Engaged site, exact skeleton match

Interstitial

No match

- 2

Bundled list, exact skeleton match

Interstitial for the 836 top-bucket names, Safety Tip for the other 7,626

A match here already has Punycode in the address bar. This check reads the display check's own top-domain result.

No match

- 3

Engaged site, one edit or adjacent swap

Safety Tip

Both names need 5 or more characters before the TLD, counted in ASCII.

No match

- 4

Top-bucket name, adjacent swap

Safety Tip

A one-edit top-bucket match only records metrics.

No match

Outcome

Proceed

No lookalike warning.

Address bar (Unicode)

apple.com

Results from a model of Chromium's navigation-time lookalike checks (main branch, July 2026). "Uses the real site" means medium or greater site engagement with the target. The model also has a spoof-check fallback between checks 2 and 3; it does not apply to these candidates. Reputation checks, remote allowlists, and redirects are not modeled.

For a lookalike that displays as Unicode, the engaged-site comparison does most of the work, because an exact match against the bundled list would already have produced

Punycode. On a cold profile, nothing compares the candidate with the target at all. Even ókta.com, whose skeleton matches the real name exactly, returns `proceed`.

The realistic phishing case is a returning user. With okta.com engaged, ókta.com gets an interstitial because its skeleton matches exactly; a skeleton one edit away would get a Safety Tip, and one two edits away, unless the difference is an adjacent swap, gets nothing. The count is of skeleton differences, not substituted characters. copy.com passes every display check, yet its skeleton is exactly copy, so a visitor who uses copy.com gets an interstitial. applø.com reaches two edits with a single substitution, because ø maps to o rather than e and its bar survives, so the near-match comparisons that run for the five-character apple find one edit too many. The model returns `proceed` for a visitor who uses apple.com daily, even though apple.com is in the bundled list's top bucket, and openai.com does the same against openai.com.

A short target needs even less. okta.com replaces k with ḳ (U+0199 , LATIN SMALL LETTER K WITH HOOK, a Hausa letter in the .com Latin table), and the hook survives into the skeleton as a combining mark, one edit from okta. Against a longer name that would earn a Safety Tip, but okta has four characters, so Chromium skips the near-match comparisons, and okta.com is not in the bundled list. The model returns `proceed` for a visitor who signs in to okta.com every day, and any brand of four characters or fewer is in the same position. Have I Been Squatted registered okta.com alongside the Cyrillic names, with four other names built on ḳ.

Each layer closes a different path: the whole-script rule needs one character off the lookalike list, the skeleton comparison catches an exact match against a known site, and the near-match comparisons catch a single surviving mark when the target is long enough. A name that passes all three has left something in its skeleton that differs from the target, and that something is drawn on screen too. Whether a reader notices it is the open question.

That combination is measurable. A run over the Tranco top 10,000 domains generated lookalikes from the same substitution sources and kept those that the model displays in Unicode and that fit a registry's IDN table, 510,333 candidates in all. Run through the navigation model on a cold profile and again with the target engaged, 95,267 drew neither an interstitial nor a Safety Tip in either case, covering 3,037 of the 10,000 targets across 64 suffixes: google.com has 24 such names, openai.com 8, and okta.com 47. None was registered or checked in a live browser; the count measures the rules, not registrations.

Navigation rules and source references

Whether applø.com could be bought

A name that displays as Unicode is only useful if a registrar will sell it, and that decision belongs to the registry that runs the TLD. IDNA sets the outer limit on which characters can appear in a domain name. Each registry then narrows that set by publishing one or more internationalized domain name (IDN) tables, usually one per script or language, which the Internet Assigned Numbers Authority (IANA) [publishes](#).

In `.com`, each IDN registration names the language or script of the label, and Verisign checks every character against the matching table. If one character is missing, the registry rejects the name. The whole label has to fit inside a single table, so a label that mixes Latin and Cyrillic usually fails here, before a browser ever sees it, and the table governs the Unicode label even when the request arrives as an `xn--` A-label. Some tables carry extra rules, such as a character allowed only next to certain others, or two characters declared variants so that registering one spelling blocks or reserves the other.

Tables limit lookalikes mainly by keeping each label inside one script and leaving out characters that no supported language needs. They cannot leave out characters that a supported language actually uses, even when those characters resemble others. `ø` is an ordinary letter in Kazakh, and Verisign's `.com` Cyrillic table includes it, which is why `applø.com` could be registered. The same holds for Latin. Verisign's `.com` Latin table (version 2.6) covers a wide range of languages, including Vietnamese, and lists 587 code points beyond ASCII, among them `î` and the Hausa letter `ƙ`. DENIC's `.de` list allows only 93 code points beyond ASCII and does not include `î`.

Table membership

The same label under different registry tables

`oktainvoicebücherappløapple`

Choose a spelling. Each row checks the complete label against one table.

<code>.com</code> Latin
Version 2.6
587 code points beyond ASCII
<code>okta.com</code>
All label characters listed
<code>.com</code> Cyrillic
Version 1.2
220 code points beyond ASCII
<code>okta.com</code>
Missing from this table: <code>okta</code>
<code>.de</code> character list
DENIC repertoire
93 code points beyond ASCII
<code>okta.de</code>
Missing from this table: <code>ƙ</code>

A dashed underline marks a character absent from that table. The suffix is context, not part of the label being checked.

The hooked letter k is Latin, but that does not put it in every Latin repertoire. It is listed in the `.com` Latin table and absent from DENIC's `.de` list.

[.com Latin source](#) [.com Cyrillic source](#) [.de character list source](#)

Character membership in the listed tables, checked September 2026. Full registry policy and availability still apply. These fixed examples are not a registration checker.

Why shorter IDN tables aren't always safer

Lookalikes already delegated in .com

If one name can be bought, the next question is how many already have been. A zone file lists delegations, the records that point resolvers from a parent zone to a domain's own name servers, and the Internet Corporation for Assigned Names and Numbers (ICANN) runs the [Centralized Zone Data Service](#) which distributes them from participating registries.

A July 2026 index of the `.com` zone held 164,184,707 delegated names, 733,362 of them in `xn--` form. Each IDN was mapped back to the ASCII names it could imitate, and a pair was kept when that ASCII name was also in the index. A September 2026 reanalysis with corrected canonicalization kept 160,721 such pairs, covering 131,095 distinct ASCII targets. One IDN can imitate more than one ASCII name, so the figure counts pairs rather than unique domains.

Dataset results

Lookalike pairs in the retained zone results

164,184,707

delegated names in the index

733,362

of those names in IDN form

The analysis retained **160,721 pairs** of delegated names.

101,246 pairs. Exact candidate match with Unicode display.

The IDN matches a candidate spelling of its ASCII name, and the model displays it in Unicode.

By script: 101,224 Latin, 14 Greek, 5 Arabic, 3 Cyrillic.

59,475 pairs. Other reverse-confusable pairs.

These come from the broader reverse comparison of similar characters.

13,201 of these are shown as Punycode by the model, including 1,167 exact candidate matches. They are part of this segment, not additional pairs.

131,095 distinct ASCII comparison names. One IDN can participate in several pairs.

Both names in each retained pair appeared in the index. Resemblance alone does not establish phishing, shared ownership, or active use.

Of those pairs, 101,246 meet two stricter conditions. The IDN matches one of the candidate spellings built for that name, and the model shows Chromium displaying it in Unicode. All but 22 are Latin. Of the rest, 14 are Greek, such as you.com, 5 are Arabic, and 3 are Cyrillic, so the Latin surface is the one visible in the zone. The other 59,475 pairs either come from a broader comparison of visually similar characters or match a candidate spelling that displays as Punycode (1,167 of them). The model shows Chromium displaying 13,201 of these 59,475 as Punycode and the other 46,274 in Unicode.

These counts describe the retained pairs, not every lookalike in .com, and a matching pair can also be a multilingual registration, a defensive registration, or an unrelated business. They say that lookalikes of this kind are routine, not which ones are malicious.

What email clients do instead

A phishing message carries the same domain in its **From** address, and there the mail client, not the browser, decides how to draw it. Chromium's check runs on hostnames the browser is about to show in the address bar. A sender address in a webmail page is ordinary text. If the mail service decodes the A-label before it sends the page, the browser receives Unicode and has nothing to check. Whether to decode the address, and whether to ask if the result is confusable, is left to each client.

The clearest result comes from five senders that are accented spellings of names in Chromium's bundled table: googlé.com, amazõñ.com, nètflix.com, coínbase.com, and bìnance.com. Each passes all seven label checks and then matches its target at stage 2, so Chromium would show all five as Punycode. Gmail showed all five in Unicode, including security@googlé.com. The browser's check, the strictest one tested, never ran.

Recorded captures and display model

How each client showed the sender domain

-

Bypass examples in this article

applø.comapplication.com

Chromium address bar (model prediction)

Unicode

Gmail Web

Unicode

Outlook Web

A-label

-

Other IDN fixtures

bücher.comпочта.com

Chromium address bar (model prediction)

Unicode

Gmail Web

Unicode

Outlook Web

A-label

-

Accented spellings of bundled-table names

googlé.comamázoñ.comnètflix.comcoínbase.comb̀inance.com

Chromium address bar (model prediction)

Punycode

Top-domain match

Gmail Web

Unicode

Outlook Web

A-label

-

Ordinary IDNs

café.comzürich.com

Chromium address bar (model prediction)

Unicode

Gmail Web

Unicode

Outlook Web

A-label

From address supplied to both web clients, test 5c541a638ea5

test@xn--80a6aa68c8d.com

Gmail WebUnicode

[Full capture](#)

From line (outlined)test@apple.com

Expanded mailed-by: fieldxn--80a6aa68c8d.com

Outlook WebA-label

Full capture

From line (outlined)test@xn--80a6aa68c8d.com

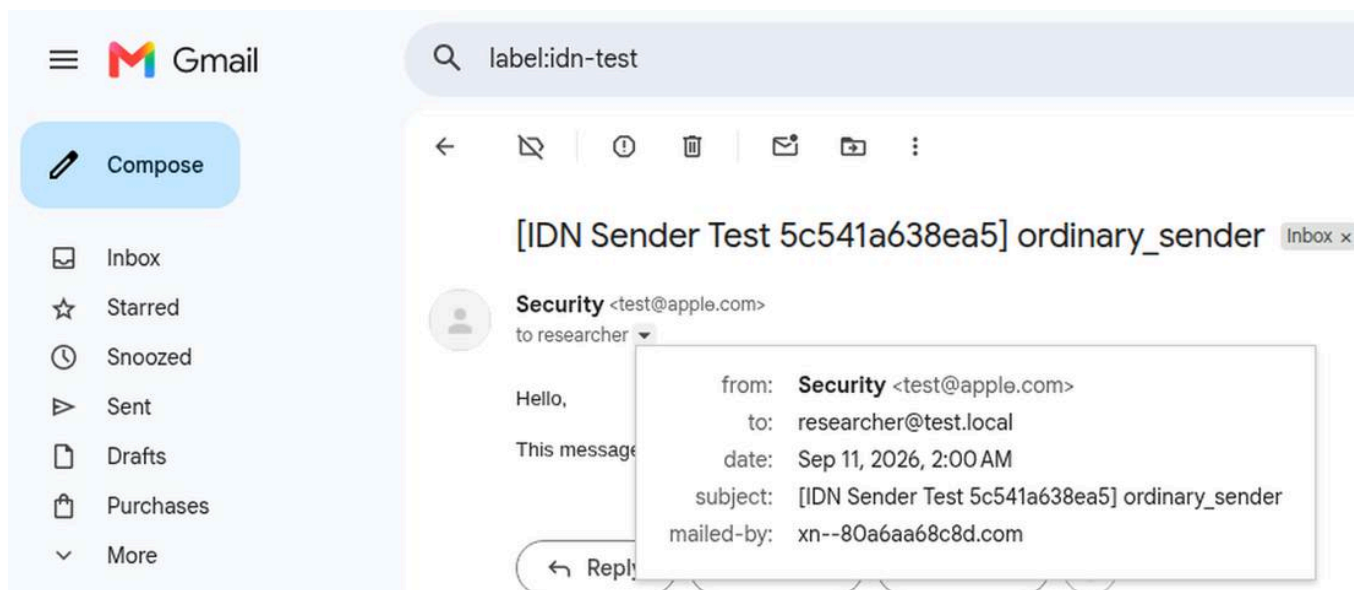
About these captures

Both images show one `sender_only` fixture per row. The comparison concerns sender presentation, not delivery, authentication, or spam classification. Native contact cards, tooltips, and Outlook's expanded details were not captured.

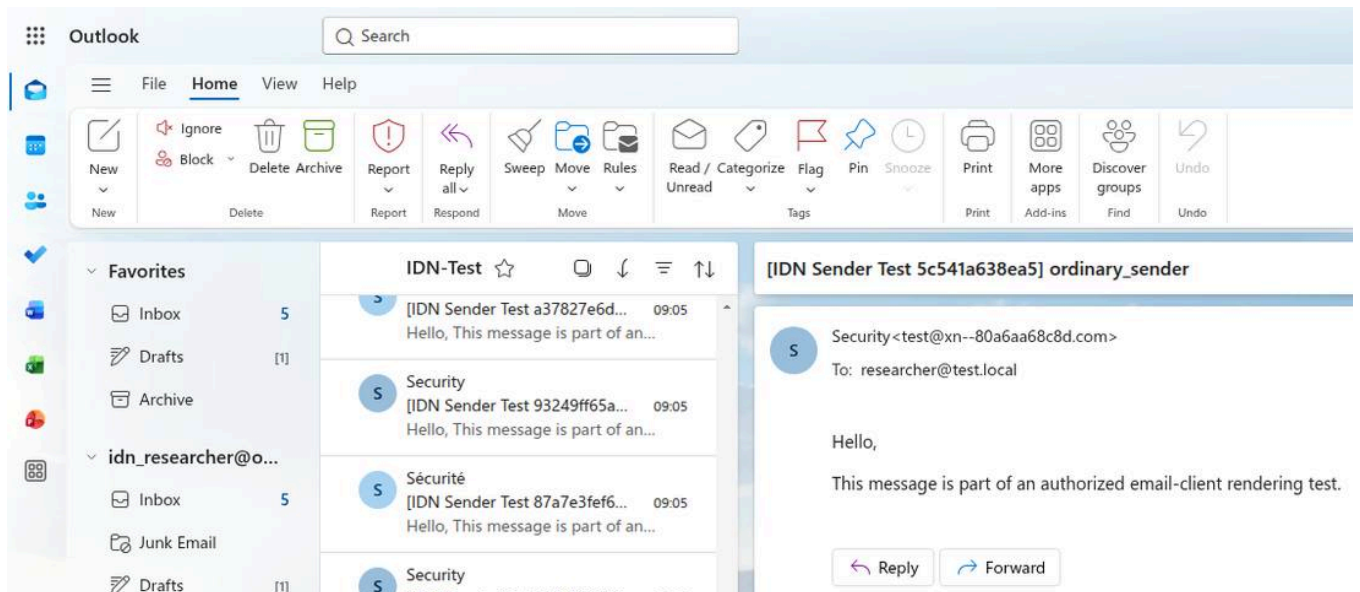
Screenshots captured on August 20 and September 26 to 27, 2026 from researcher-owned mailboxes; the Chromium column is the display model's prediction, not a capture. Client builds were not recorded.

The result did not depend on the name. Each web client received the same 20 valid internationalized senders and two ASCII controls. Gmail showed every one in Unicode, and Outlook Web showed every one as an A-label. Gmail's expanded details keep `from:` in Unicode; the Punycode form appears only in a separate `mailed-by:` field, which is not the address a reader checks. Outlook Web goes the other way. `café.com` and `zürich.com` resemble nothing in particular, and Chromium would show both in Unicode, yet Outlook Web printed their A-labels alongside the lookalikes. Neither client separated a spelling built to imitate another name from an ordinary one. One decoded everything, and the other decoded nothing.

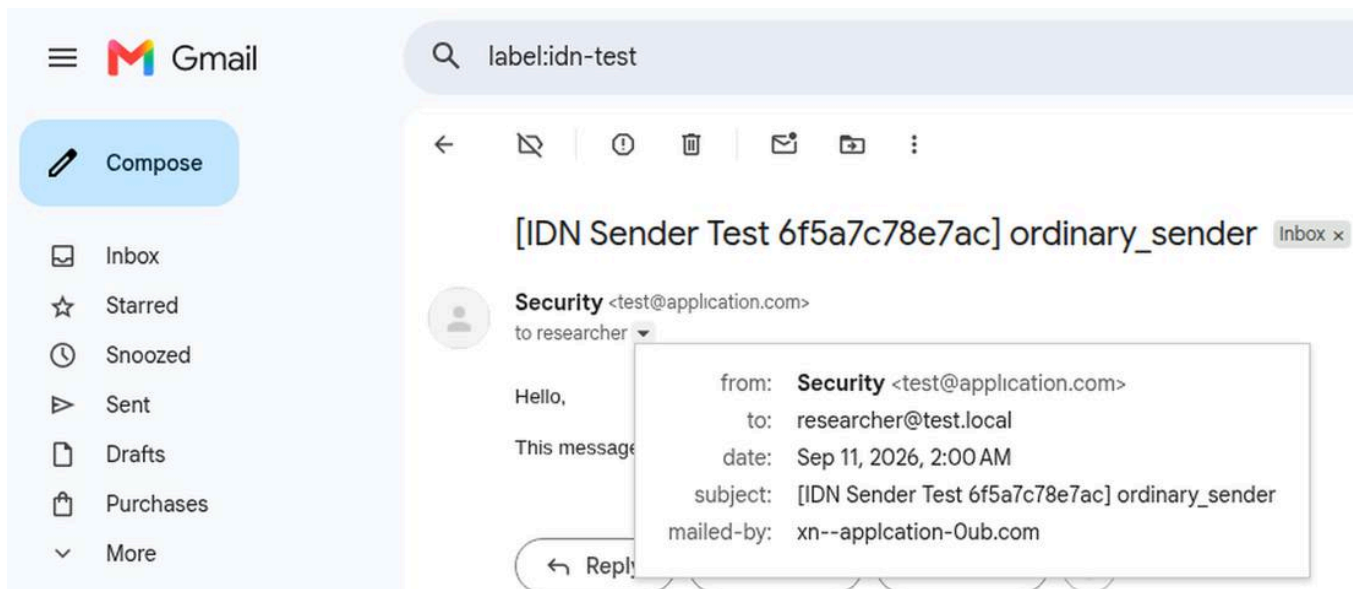
A second run in late September put the apple names themselves in front of both web clients, along with ten other domains. Gmail showed `test@apple.com` in Unicode in the From line and in the expanded details, with the A-label relegated to `mailed-by:`. It showed Zheng's `apple.com`, which Chrome has shown as Punycode since 2017, the same way, and it did the same for `application.com`, a dotless-ı spelling of `application.com`. Outlook Web printed `test@xn--80a6aa68c8d.com` and `test@xn--application-0ub.com`. Neither client ran a lookalike check.



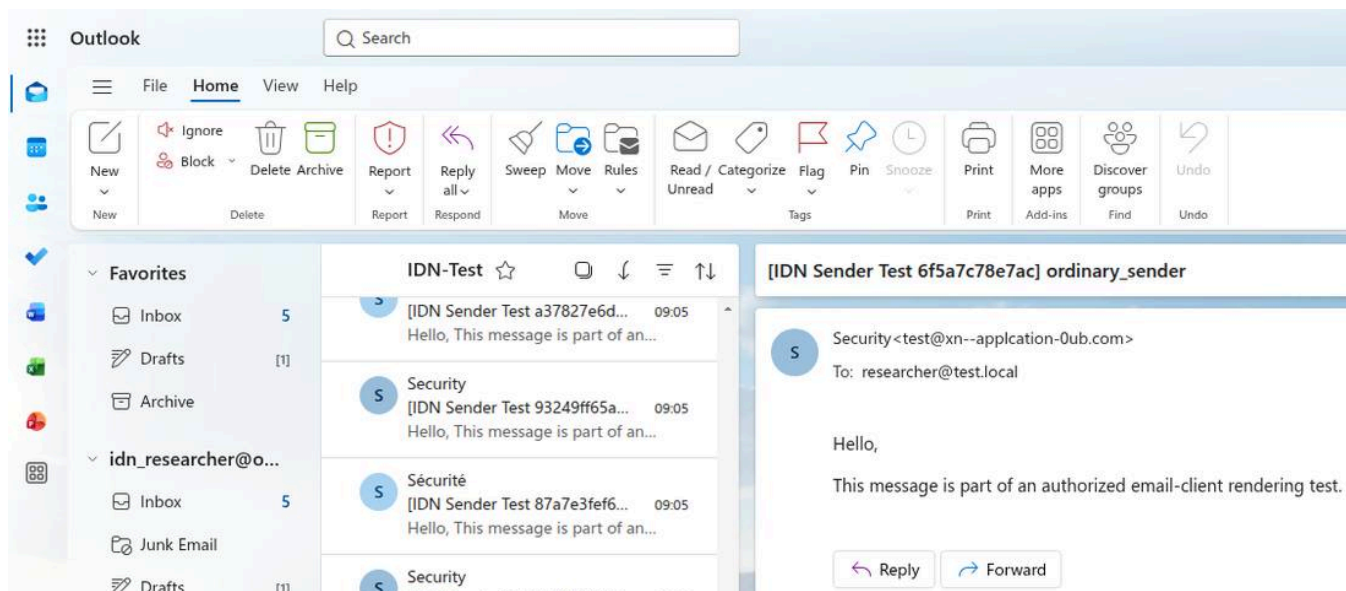
Gmail, 26 September 2026. The From line and expanded details show `apple.com` in Unicode. Only `mailed-by` shows the A-label.



Outlook Web, 27 September 2026. The same message, with the sender printed as its A-label.



Gmail, 26 September 2026. application.com, a dotless-i spelling, shown in Unicode.



Outlook Web, 27 September 2026. The same message, with the sender printed as its A-label.

Previous slide Next slide

Each fixture put the domain in the `From` header as an A-label, the form mail headers traditionally require, and was imported through each provider's application programming interface (API) into a researcher-owned mailbox. The recorded result is the address the opened message showed. Spam filtering, sender reputation, and link scanning are separate systems and were not measured; these fixtures test how each client draws the address, not how convincing it is.

No one layer stops these names. Chrome has the strictest check and still shows them, Gmail and Outlook disagree with Chrome and with each other, and browser updates only move the line, as Chrome 148 did for `p` and `ж`.

Register the variants that survive a skeleton, and monitor for the rest. Accented spellings are the ones Chrome already catches. The ones that reach a reader use letters like `ø`, `γ`, and `к`, or target a name outside Chrome's list, which is almost every name. A brand of four characters or fewer gets no near-match protection at all, so register its hooked and stroked spellings directly; the Tranco run found 47 spellings of `okta.com` that draw no warning. Monitor new `xn--` registrations within two edits of each protected domain.

Check senders at the mail gateway. Gmail shows every IDN sender in Unicode with no lookalike check, and SPF, DKIM, and DMARC all pass for a lookalike the attacker owns. Run the skeleton comparison once at the gateway, against the organization's own domains and its suppliers', and keep external-sender banners on.

Use passkeys or FIDO2. A credential registered on apple.com will not answer a login page on xn--80a6aa68c8d.com, whatever the address bar shows.

-

Treat xn-- as a signal at DNS and the proxy. Most organizations rarely resolve or mail IDN domains. Log, banner, or block xn-- names outside an allowlist; this works on the encoded name, which no display choice can hide. In Firefox, `network.IDN_show_punycode` set to `true` shows every IDN as xn--.

The candidates and zone matches in this post are leads, not verdicts. Who owns a name, where it is hosted, and whether it is active decide which ones need action.

What this does and does not establish

The names Have I Been Squatted registered were checked live. Each hosts the Have I Been Squatted landing page, and Chrome shows each in Unicode. Everything else, the other candidates, the navigation predictions, and the zone counts, rests on the Python model, and the ICU version, the lookalike lists, the candidate set, and the bundled domain file all change its output. Each figure is labeled with the evidence behind it, and the accordions below say exactly what each kind of evidence can and cannot support.

Browser model and character selection

Registry tables and zone data

Email fixtures and screenshots

The checks stay separate

Traced back through the layers, apple.com passed each one for a different reason. Verisign sold it because ø is a Kazakh letter on the .com Cyrillic table. Chromium's per-label check passed it because ø is allowed and not on the lookalike list. The skeleton comparison passed it because ø leaves a bar behind, so the skeleton is not apple. The navigation throttle passed it because two edits is one more than it looks for. Gmail would draw it in Unicode without running any of those checks, and Outlook Web would draw it as xn--80a6aa68c8d.com without running them either. Each of those decisions is correct on its own terms, and none of them knows what the others decided.

None of it changes which domain the name identifies. ICU and Chromium's confusable data will keep moving; the characters that work in Chrome 155 will not all work in Chrome 165, and others will. A Unicode spelling never identifies the registrant behind it. The one comparison that holds up across every layer is whether the full domain matches one already known to belong to the organization claiming it. No browser or mail client makes that comparison on the reader's behalf.

Domain protection

Detect adversary infrastructure while it is being staged.

Have I Been Squatted helps security teams detect lookalike domains, certificate and DNS changes, and staging infrastructure, investigate the evidence, and coordinate takedowns.

[Start free trial](#)[Explore domain protection](#)

Archived from <https://haveibeenquatted.com/blog/turning-idn-edge-cases-into-typosquats>. Rendered offline from the local Markdown copy.